Swarm 설정 및 사용 예제

개요

이 문서는 Rust libp2p의 Swarm 컴포넌트를 설정하고 사용하는 다양한 방법을 실제 코드 예제와 함께 제공합니다. Swarm은 libp2p의 핵심 컴포넌트로, 네트워크 연결, 프로토콜 관리, 이벤트 처리를 통합적으로 관리합니다.

기본 Swarm 설정

1. 최소한의 Swarm 설정

```
use libp2p::{
noise, ping, tcp, yamux,
swarm::{SwarmBuilder, SwarmEvent},
Multiaddr, PeerId,
};
use std::{error::Error, time::Duration};
use tokio;

#[tokio::main]
async fn main() -> Result<(), Box<dyn Error>> {
// 기본 Swarm 생성
let mut swarm = SwarmBuilder::with_new_identity()
```

```
.with_tokio()
.with_tcp(
tcp::Config::default(),
noise::Config::new,
yamux::Config::default,
)?
.with_behaviour(|_| ping::Behaviour::default())?
.build();


// 로컬 주소에서 리스닝 시작
swarm.listen_on("/ip4/0.0.0.0/tcp/0".parse()?)?;


// 이벤트 루프
loop {
match swarm.select_next_some().await {
SwarmEvent::NewListenAddr { address, .. } => {
println!("Listening on {address:?}");
}
SwarmEvent::Behaviour(event) => {
println!("Ping event: {event:?}");
}
_ => {}
}
```

```
}

}
```

2. 고급 Swarm 설정

```
use libp2p::{

identify, noise, ping, tcp, yamux,

swarm::{NetworkBehaviour, SwarmBuilder, SwarmEvent, Config as SwarmConfig},

Multiaddr, PeerId,

};

use std::{error::Error, time::Duration};


// 복합 NetworkBehaviour 정의

#[derive(NetworkBehaviour)]

struct MyBehaviour {

ping: ping::Behaviour,

identify: identify::Behaviour,

}


#[tokio::main]

async fn main() -> Result<(), Box<dyn Error>> {

let mut swarm = SwarmBuilder::with_new_identity()

.with_tokio()
```

```
.with_tcp(
tcp::Config::default().port_reuse(true),
noise::Config::new,
yamux::Config::default(),
)?
.with_quic() // QUIC 전송 추가
.with_behaviour(|key| {
Ok(MyBehaviour {
ping: ping::Behaviour::default(),
identify: identify::Behaviour::new(
identify::Config::new("/my-protocol/1.0.0".to_string(), key.public())
.with_agent_version("my-app/1.0.0".to_string()),
),
})
})?
.with_swarm_config(|cfg| {
cfg.with_idle_connection_timeout(Duration::from_secs(30))
.with_max_negotiating_inbound_streams(100)
.with_per_connection_event_buffer_size(1024)
})
.build();


// 여러 주소에서 리스닝
```

```
swarm.listen_on("/ip4/0.0.0.0/tcp/0".parse()?)?;

swarm.listen_on("/ip4/0.0.0.0/udp/0/quic-v1".parse()?)?;



// 이벤트 처리

loop {

match swarm.select_next_some().await {

SwarmEvent::NewListenAddr { address, .. } => {

println!("Listening on {address:?}");

}

SwarmEvent::Behaviour(MyBehaviourEvent::Ping(event)) => {

println!("Ping: {event:?}");

}

SwarmEvent::Behaviour(MyBehaviourEvent::Identify(event)) => {

println!("Identify: {event:?}");

}

SwarmEvent::ConnectionEstablished { peer_id, endpoint, .. } => {

println!("Connected to {peer_id} via {endpoint:?}");

}

SwarmEvent::ConnectionClosed { peer_id, cause, .. } => {

println!("Disconnected from {peer_id}, cause: {cause:?}");

}

_ => {}

}
```

```
}

}
```

전송 계층 설정 예제

1. TCP 전송 설정

```
use libp2p::{tcp, noise, yamux, swarm::SwarmBuilder};

use std::time::Duration;


async fn create_tcp_swarm() -> Result<libp2p::Swarm<ping::Behaviour>, Box<dyn st

let swarm = SwarmBuilder::with_new_identity()

.with_tokio()

.with_tcp(

tcp::Config::default()

.port_reuse(true)

.nodelay(true)

.ttl(64),

noise::Config::new,

yamux::Config::default()

.with_max_buffer_size(16 * 1024 * 1024)

.with_max_num_streams(256),

)?
```

```
.with_behaviour(|_| ping::Behaviour::default())?

.build();

Ok(swarm)

}
```

2. QUIC 전송 설정

```
use libp2p::{quic, swarm::SwarmBuilder};


async fn create_quic_swarm() -> Result<libp2p::Swarm<ping::Behaviour>, Box<dyn s

let swarm = SwarmBuilder::with_new_identity()

.with_tokio()

.with_quic()

.with_behaviour(|_| ping::Behaviour::default())?

.build();

Ok(swarm)

}
```

3. 다중 전송 프로토콜 설정

```
use libp2p::{tcp, quic, websocket, noise, yamux, swarm::SwarmBuilder};
```

```
async fn create_multi_transport_swarm() -> Result<libp2p::Swarm<ping::Behaviour>

let swarm = SwarmBuilder::with_new_identity()

.with_tokio()

.with_tcp(

tcp::Config::default(),

noise::Config::new,

yamux::Config::default(),

)?

.with_quic()

.with_websocket(

(noise::Config::new, yamux::Config::default),

).await?

.with_behaviour(|_| ping::Behaviour::default())?

.build();

Ok(swarm)

}
```

프로토콜별 설정 예제

1. Kademlia DHT 설정

```
use libp2p::{

kad, noise, tcp, yamux,
```

```
swarm::{NetworkBehaviour, SwarmBuilder},

PeerId, Multiaddr,

};

use std::collections::hash_map::DefaultHasher;

use std::hash::{Hash, Hasher};



#[derive(NetworkBehaviour)]

struct KadBehaviour {

kad: kad::Behaviour<kad::store::MemoryStore>,

}



async fn create_kad_swarm() -> Result<libp2p::Swarm<KadBehaviour>, Box<dyn std::

let mut swarm = SwarmBuilder::with_new_identity()

.with_tokio()

.with_tcp(

tcp::Config::default(),

noise::Config::new,

yamux::Config::default(),

)?

.with_behaviour(|key| {

let peer_id = key.public().to_peer_id();

let store = kad::store::MemoryStore::new(peer_id);

let kad_config = kad::Config::default();
```

```
Ok(KadBehaviour {
kad: kad::Behaviour::with_config(peer_id, store, kad_config),
})
})?
.build();

// Bootstrap 노드 추가
if let Some(bootstrap_addr) = get_bootstrap_address() {
swarm.behaviour_mut().kad.add_address(&bootstrap_addr.0, bootstrap_addr.1);
swarm.behaviour_mut().kad.bootstrap().ok();
}

Ok(swarm)
}

fn get_bootstrap_address() -> Option<(PeerId, Multiaddr)> {
// 실제 구현에서는 알려진 bootstrap 노드 주소를 반환
None
}
```

2. Gossipsub 설정

```
use libp2p::{
```

```
gossipsub, mdns, noise, tcp, yamux,

swarm::{NetworkBehaviour, SwarmBuilder},

};

use std::{

collections::hash_map::DefaultHasher,

hash::{Hash, Hasher},

time::Duration,

};



#[derive(NetworkBehaviour)]

struct GossipBehaviour {

gossipsub: gossipsub::Behaviour,

mdns: mdns::tokio::Behaviour,

}



async fn create_gossip_swarm() -> Result<libp2p::Swarm<GossipBehaviour>, Box<dyn

let swarm = SwarmBuilder::with_new_identity()

.with_tokio()

.with_tcp(

tcp::Config::default(),

noise::Config::new,

yamux::Config::default(),

)?
```

```
.with_behaviour(|key| {
// 메시지 ID 함수 정의
let message_id_fn = |message: &gossipsub::Message| {
let mut s = DefaultHasher::new();
message.data.hash(&mut s);
gossipsub::MessageId::from(s.finish().to_string())
};

// Gossipsub 설정
let gossipsub_config = gossipsub::ConfigBuilder::default()
.heartbeat_interval(Duration::from_secs(10))
.validation_mode(gossipsub::ValidationMode::Strict)
.message_id_fn(message_id_fn)
.max_transmit_size(262144)
.build()
.expect("Valid config");

let gossipsub = gossipsub::Behaviour::new(
gossipsub::MessageAuthenticity::Signed(key.clone()),
gossipsub_config,
)?;

let mdns = mdns::tokio::Behaviour::new(
```

```
mdns::Config::default(),
key.public().to_peer_id(),
)?;

Ok(GossipBehaviour { gossipsub, mdns })
})?
.build();

Ok(swarm)
}
```

3. Request-Response 프로토콜 설정

```
use libp2p::{
request_response::{self, ProtocolSupport, ResponseChannel},
noise, tcp, yamux,
swarm::{NetworkBehaviour, SwarmBuilder},
StreamProtocol,
};
use serde::{Deserialize, Serialize};

#[derive(Debug, Clone, Serialize, Deserialize)]
```

```
struct MyRequest {
data: String,
}

#[derive(Debug, Clone, Serialize, Deserialize)]
struct MyResponse {
result: String,
}

#[derive(NetworkBehaviour)]
struct RequestResponseBehaviour {
request_response: request_response::Behaviour<request_response::cbor::Codec<MyRe
}

async fn create_request_response_swarm() -> Result<libp2p::Swarm<RequestResponse
let swarm = SwarmBuilder::with_new_identity()
.with_tokio()
.with_tcp(
tcp::Config::default(),
noise::Config::new,
yamux::Config::default(),
)?
.with_behaviour(|_| {
```

```
let protocols = std::iter::once((
StreamProtocol::new("/my-protocol/1.0.0"),
ProtocolSupport::Full,
));
let cfg = request_response::Config::default();
let behaviour = request_response::Behaviour::with_codec(
request_response::cbor::Codec::default(),
protocols,
cfg,
);


Ok(RequestResponseBehaviour {
request_response: behaviour,
})
})?
.build();


Ok(swarm)
}
```

연결 관리 예제

1. 연결 제한 설정

```
use libp2p::{
connection_limits::{self, ConnectionLimits},
core::upgrade::Version,
noise, tcp, yamux,
swarm::{NetworkBehaviour, SwarmBuilder},
};


#[derive(NetworkBehaviour)]
struct LimitedBehaviour {
ping: ping::Behaviour,
connection_limits: connection_limits::Behaviour,
}


async fn create_limited_swarm() -> Result<libp2p::Swarm<LimitedBehaviour>, Box<d
let swarm = SwarmBuilder::with_new_identity()
.with_tokio()
.with_tcp(
tcp::Config::default(),
noise::Config::new,
yamux::Config::default(),
)?
.with_behaviour(|_| {
```

```
let limits = ConnectionLimits::default()
.with_max_pending_incoming(Some(10))
.with_max_pending_outgoing(Some(20))
.with_max_established_incoming(Some(100))
.with_max_established_outgoing(Some(200))
.with_max_established_per_peer(Some(5));


Ok(LimitedBehaviour {
ping: ping::Behaviour::default(),
connection_limits: connection_limits::Behaviour::new(limits),
})
})?
.build();


Ok(swarm)
}
```

2. 자동 연결 관리

```
use libp2p::{
autonat, identify, ping,
swarm::{NetworkBehaviour, SwarmBuilder, SwarmEvent},
PeerId, Multiaddr,
```

```
};
use std::collections::HashMap;


#[derive(NetworkBehaviour)]
struct AutoConnectBehaviour {
ping: ping::Behaviour,
identify: identify::Behaviour,
autonat: autonat::Behaviour,
}


struct ConnectionManager {
swarm: libp2p::Swarm<AutoConnectBehaviour>,
known_peers: HashMap<PeerId, Vec<Multiaddr>>,
reconnect_attempts: HashMap<PeerId, u32>,
}


impl ConnectionManager {
async fn new() -> Result<Self, Box<dyn std::error::Error>> {
let swarm = SwarmBuilder::with_new_identity()
.with_tokio()
.with_tcp(
tcp::Config::default(),
noise::Config::new,
```

```
yamux::Config::default(),

)?

.with_behaviour(|key| {

Ok(AutoConnectBehaviour {

ping: ping::Behaviour::default(),

identify: identify::Behaviour::new(

identify::Config::new("/my-protocol/1.0.0".to_string(), key.public())

),

autonat: autonat::Behaviour::new(

key.public().to_peer_id(),

autonat::Config::default(),

),

})

})?

.build();



Ok(ConnectionManager {

swarm,

known_peers: HashMap::new(),

reconnect_attempts: HashMap::new(),

})

}
```

```
async fn run(&mut self) {
loop {
match self.swarm.select_next_some().await {
SwarmEvent::ConnectionClosed { peer_id, cause, .. } => {
println!("Connection to {peer_id} closed: {cause:?}");
self.handle_disconnection(peer_id).await;
}
SwarmEvent::Behaviour(AutoConnectBehaviourEvent::Identify(identify::Event::Recei
peer_id,
info,
..
})) => {
// 피어 정보 저장
self.known_peers.insert(peer_id, info.listen_addrs);
}
SwarmEvent::Behaviour(AutoConnectBehaviourEvent::Autonat(autonat::Event::StatusC
old,
new,
})) => {
println!("NAT status changed from {old:?} to {new:?}");
}
_ => {}
}
```

```
}
}

async fn handle_disconnection(&mut self, peer_id: PeerId) {
let attempts = self.reconnect_attempts.entry(peer_id).or_insert(0);
if *attempts < 3 {
*attempts += 1;
if let Some(addrs) = self.known_peers.get(&peer_id) {
for addr in addrs {
if let Err(e) = self.swarm.dial(addr.clone()) {
println!("Failed to redial {peer_id}: {e}");
} else {
println!("Attempting to reconnect to {peer_id}");
break;
}
}
}
} else {
println!("Max reconnection attempts reached for {peer_id}");
self.reconnect_attempts.remove(&peer_id);
}
}
```

```
}
```

이벤트 처리 패턴

1. 기본 이벤트 처리

```
use libp2p::swarm::{SwarmEvent, ConnectionError};
use std::collections::HashMap;

struct EventHandler {
connected_peers: HashMap<PeerId, u32>,
}

impl EventHandler {
fn new() -> Self {
Self {
connected_peers: HashMap::new(),
}
}

async fn handle_swarm_event(&mut self, event: SwarmEvent<MyBehaviourEvent>) {
match event {
SwarmEvent::ConnectionEstablished {
```

```
peer_id,
connection_id,
endpoint,
num_established,
established_in,
..
} => {
println!(
"Connected to {peer_id} (connection {connection_id}) via {endpoint:?} \
in {established_in:?}. Total connections: {num_established}"
);
self.connected_peers.insert(peer_id, num_established.get());
}
SwarmEvent::ConnectionClosed {
peer_id,
connection_id,
endpoint,
num_established,
cause,
..
} => {
println!(
"Disconnected from {peer_id} (connection {connection_id}) via {endpoint:?}. \
```

```
Remaining connections: {num_established}"
);
if num_established == 0 {
self.connected_peers.remove(&peer_id);
} else {
self.connected_peers.insert(peer_id, num_established);
}


// 연결 종료 원인 분석
if let Some(ConnectionError::IO(io_error)) = cause {
println!("Connection closed due to IO error: {io_error}");
}
}
SwarmEvent::NewListenAddr { listener_id, address } => {
println!("Listening on {address} (listener {listener_id:?})");
}
SwarmEvent::ExpiredListenAddr { listener_id, address } => {
println!("No longer listening on {address} (listener {listener_id:?})");
}
SwarmEvent::OutgoingConnectionError { peer_id, connection_id, error } => {
println!("Failed to connect to {peer_id:?} (connection {connection_id}): {error}
}
SwarmEvent::IncomingConnectionError { connection_id, local_addr, send_back_addr,
```

```
println!(
"Failed incoming connection {connection_id} from {send_back_addr} \
to {local_addr}: {error}"
);
}
SwarmEvent::Behaviour(behaviour_event) => {
self.handle_behaviour_event(behaviour_event).await;
}
_ => {}
}
}


async fn handle_behaviour_event(&mut self, event: MyBehaviourEvent) {
// 프로토콜별 이벤트 처리
match event {
MyBehaviourEvent::Ping(ping_event) => {
println!("Ping event: {ping_event:?}");
}
MyBehaviourEvent::Identify(identify_event) => {
println!("Identify event: {identify_event:?}");
}
// 기타 프로토콜 이벤트 처리...
}
```

```
}

}
```

2. 비동기 이벤트 처리

```
use tokio::{select, time::{interval, Duration}};

use futures::stream::StreamExt;


struct AsyncEventHandler {

swarm: libp2p::Swarm<MyBehaviour>,

periodic_tasks: tokio::time::Interval,

}


impl AsyncEventHandler {

fn new(swarm: libp2p::Swarm<MyBehaviour>) -> Self {

Self {

swarm,

periodic_tasks: interval(Duration::from_secs(30)),

}

}


async fn run(&mut self) {

loop {
```

```
select! {
// Swarm 이벤트 처리
event = self.swarm.select_next_some() => {
self.handle_swarm_event(event).await;
}
// 주기적 작업 실행
_ = self.periodic_tasks.tick() => {
self.perform_periodic_tasks().await;
}
// 외부 명령 처리 (예: 사용자 입력)
command = self.receive_command() => {
if let Some(cmd) = command {
self.handle_command(cmd).await;
}
}
}
}
}
```

```
async fn handle_swarm_event(&mut self, event: SwarmEvent<MyBehaviourEvent>) {
// 이벤트 처리 로직
}
```

```
async fn perform_periodic_tasks(&mut self) {
// 주기적 작업 (예: 연결 상태 점검, 메트릭 수집 등)
println!("Performing periodic maintenance tasks...");
// 연결된 피어 수 확인
let connected_peers = self.swarm.connected_peers().count();
println!("Currently connected to {connected_peers} peers");
// 리스닝 주소 확인
for addr in self.swarm.listeners() {
println!("Listening on: {addr}");
}
}

async fn receive_command(&mut self) -> Option<Command> {
// 외부 명령 수신 로직 (예: 채널, 웹소켓 등)
None
}

async fn handle_command(&mut self, command: Command) {
match command {
Command::Dial(addr) => {
if let Err(e) = self.swarm.dial(addr) {
println!("Failed to dial: {e}");
}
```

```
}
Command::Disconnect(peer_id) => {
self.swarm.disconnect_peer_id(peer_id).ok();
}
// 기타 명령 처리...
}
}
}

enum Command {
Dial(Multiaddr),
Disconnect(PeerId),
}
```

성능 최적화 설정

1. 메모리 최적화

```
use libp2p::{
swarm::{Config as SwarmConfig, SwarmBuilder},
yamux, tcp, noise,
};
```

```
async fn create_optimized_swarm() -> Result<libp2p::Swarm<MyBehaviour>, Box<dyn

let swarm = SwarmBuilder::with_new_identity()

.with_tokio()

.with_tcp(

tcp::Config::default()

.nodelay(true)

.ttl(64),

noise::Config::new,

yamux::Config::default()

.with_max_buffer_size(1024 * 1024) // 1MB 버퍼

.with_max_num_streams(128) // 최대 스트림 수 제한

.with_window_update_mode(yamux::WindowUpdateMode::OnRead),

)?

.with_behaviour(|_| Ok(MyBehaviour::default()))?

.with_swarm_config(|cfg| {

cfg.with_idle_connection_timeout(Duration::from_secs(60))

.with_max_negotiating_inbound_streams(32)

.with_per_connection_event_buffer_size(256)

})

.build();



Ok(swarm)
```

```
}
```

2. 네트워크 최적화

```
use libp2p::{
tcp, quic, noise, yamux,
swarm::SwarmBuilder,
};

async fn create_high_performance_swarm() -> Result<libp2p::Swarm<MyBehaviour>, B
let swarm = SwarmBuilder::with_new_identity()
.with_tokio()
.with_tcp(
tcp::Config::default()
.nodelay(true) // TCP_NODELAY 활성화
.port_reuse(true) // 포트 재사용
.ttl(64), // TTL 설정
noise::Config::new,
yamux::Config::default()
.with_max_buffer_size(16 * 1024 * 1024) // 16MB 버퍼
.with_max_num_streams(512) // 높은 스트림 한계
.with_window_update_mode(yamux::WindowUpdateMode::OnReceive),
)?
```

```
.with_quic() // QUIC 추가로 성능 향상
.with_behaviour(|_| Ok(MyBehaviour::default()))?
.with_swarm_config(|cfg| {
cfg.with_idle_connection_timeout(Duration::from_secs(300)) // 긴 타임아웃
.with_max_negotiating_inbound_streams(128)
.with_per_connection_event_buffer_size(1024)
})
.build();

Ok(swarm)
}
```

오류 처리 및 복구

1. 연결 오류 처리

```
use libp2p::swarm::{DialError, ListenError, ConnectionError};

struct ErrorHandler;

impl ErrorHandler {
fn handle_dial_error(&self, peer_id: Option<PeerId>, error: DialError) {
match error {
```

```
DialError::Transport(transport_errors) => {
println!("Transport errors when dialing {peer_id:?}:");
for (addr, error) in transport_errors {
println!(" {addr}: {error}");
}
}
DialError::NoAddresses => {
println!("No addresses available for {peer_id:?}");
}
DialError::LocalPeerId => {
println!("Attempted to dial local peer ID");
}
DialError::Aborted => {
println!("Dial aborted for {peer_id:?}");
}
DialError::WrongPeerId { obtained, endpoint } => {
println!("Wrong peer ID: expected {peer_id:?}, got {obtained} at {endpoint:?}");
}
DialError::Denied { cause } => {
println!("Dial denied for {peer_id:?}: {cause}");
}
}
}
```

```
fn handle_listen_error(&self, error: ListenError) {
match error {
ListenError::Transport(transport_error) => {
println!("Transport error while listening: {transport_error}");
}
ListenError::Aborted => {
println!("Listen operation aborted");
}
ListenError::Denied { cause } => {
println!("Listen denied: {cause}");
}
}
}

fn handle_connection_error(&self, error: ConnectionError<std::io::Error>) {
match error {
ConnectionError::IO(io_error) => {
println!("IO error in connection: {io_error}");
}
ConnectionError::Handler(handler_error) => {
println!("Handler error in connection: {handler_error}");
}
```

```
}
}
}
```

2. 자동 복구 메커니즘

```
use std::collections::HashMap;
use tokio::time::{sleep, Duration};

struct RecoveryManager {
swarm: libp2p::Swarm<MyBehaviour>,
failed_connections: HashMap<PeerId, FailureInfo>,
bootstrap_nodes: Vec<(PeerId, Multiaddr)>,
}

struct FailureInfo {
count: u32,
last_attempt: std::time::Instant,
backoff_duration: Duration,
}

impl RecoveryManager {
fn new(swarm: libp2p::Swarm<MyBehaviour>) -> Self {
```

```
Self {
swarm,
failed_connections: HashMap::new(),
bootstrap_nodes: vec![
// 알려진 bootstrap 노드들
],
}
}

async fn handle_connection_failure(&mut self, peer_id: PeerId) {
let failure_info = self.failed_connections
.entry(peer_id)
.or_insert(FailureInfo {
count: 0,
last_attempt: std::time::Instant::now(),
backoff_duration: Duration::from_secs(1),
});

failure_info.count += 1;
failure_info.last_attempt = std::time::Instant::now();
// 지수 백오프
failure_info.backoff_duration = std::cmp::min(
failure_info.backoff_duration * 2,
```

```
Duration::from_secs(300), // 최대 5분
);

// 최대 재시도 횟수 확인
if failure_info.count <= 5 {
println!("Scheduling reconnection to {peer_id} in {:?}", failure_info.backoff_du
let backoff = failure_info.backoff_duration;
tokio::spawn(async move {
sleep(backoff).await;
// 재연결 시도 로직
});
} else {
println!("Max retry attempts reached for {peer_id}, removing from retry list");
self.failed_connections.remove(&peer_id);
}
}

async fn ensure_connectivity(&mut self) {
let connected_count = self.swarm.connected_peers().count();
if connected_count < 3 { // 최소 연결 수 유지
println!("Low connectivity ({connected_count} peers), attempting to connect to b
for (peer_id, addr) in &self.bootstrap_nodes {
if !self.swarm.is_connected(peer_id) {
```

```
if let Err(e) = self.swarm.dial(addr.clone()) {
println!("Failed to dial bootstrap node {peer_id}: {e}");
}
}
}
}
}
}
```

모니터링 및 메트릭

1. 기본 메트릭 수집

```
use std::sync::atomic::{AtomicU64, Ordering};
use std::sync::Arc;

#[derive(Clone)]
struct SwarmMetrics {
connections_established: Arc<AtomicU64>,
connections_closed: Arc<AtomicU64>,
bytes_sent: Arc<AtomicU64>,
bytes_received: Arc<AtomicU64>,
events_processed: Arc<AtomicU64>,
```

```
}

impl SwarmMetrics {
fn new() -> Self {
Self {
connections_established: Arc::new(AtomicU64::new(0)),
connections_closed: Arc::new(AtomicU64::new(0)),
bytes_sent: Arc::new(AtomicU64::new(0)),
bytes_received: Arc::new(AtomicU64::new(0)),
events_processed: Arc::new(AtomicU64::new(0)),
}
}

fn record_connection_established(&self) {
self.connections_established.fetch_add(1, Ordering::Relaxed);
}

fn record_connection_closed(&self) {
self.connections_closed.fetch_add(1, Ordering::Relaxed);
}

fn record_bytes_sent(&self, bytes: u64) {
self.bytes_sent.fetch_add(bytes, Ordering::Relaxed);
```

```
}

fn record_bytes_received(&self, bytes: u64) {
self.bytes_received.fetch_add(bytes, Ordering::Relaxed);
}

fn record_event_processed(&self) {
self.events_processed.fetch_add(1, Ordering::Relaxed);
}

fn get_stats(&self) -> SwarmStats {
SwarmStats {
connections_established: self.connections_established.load(Ordering::Relaxed),
connections_closed: self.connections_closed.load(Ordering::Relaxed),
bytes_sent: self.bytes_sent.load(Ordering::Relaxed),
bytes_received: self.bytes_received.load(Ordering::Relaxed),
events_processed: self.events_processed.load(Ordering::Relaxed),
}
}
}

struct SwarmStats {
connections_established: u64,
```

```
connections_closed: u64,
bytes_sent: u64,
bytes_received: u64,
events_processed: u64,
}


impl std::fmt::Display for SwarmStats {
fn fmt(&self, f: &mut std::fmt::Formatter<'_>) -> std::fmt::Result {
write!(f,
"Connections: {} established, {} closed | \
Traffic: {} bytes sent, {} bytes received | \
Events: {} processed",
self.connections_established,
self.connections_closed,
self.bytes_sent,
self.bytes_received,
self.events_processed
)
}
}
```

2. 성능 모니터링

```rust
use std::time::{Duration, Instant};

use tokio::time::interval;


struct PerformanceMonitor {

metrics: SwarmMetrics,

last_stats: SwarmStats,

last_check: Instant,

}


impl PerformanceMonitor {

fn new(metrics: SwarmMetrics) -> Self {

let stats = metrics.get_stats();

Self {

metrics,

last_stats: stats,

last_check: Instant::now(),

}

}


async fn start_monitoring(&mut self) {

let mut interval = interval(Duration::from_secs(30));

loop {
```

```
interval.tick().await;
self.report_performance().await;
}
}

async fn report_performance(&mut self) {
let current_stats = self.metrics.get_stats();
let now = Instant::now();
let duration = now.duration_since(self.last_check);

let connections_per_sec = (current_stats.connections_established - self.last_sta
/ duration.as_secs_f64();
let bytes_per_sec = (current_stats.bytes_sent + current_stats.bytes_received
- self.last_stats.bytes_sent - self.last_stats.bytes_received) as f64
/ duration.as_secs_f64();
let events_per_sec = (current_stats.events_processed - self.last_stats.events_pr
/ duration.as_secs_f64();

println!("Performance Report:");
println!(" Connections/sec: {:.2}", connections_per_sec);
println!(" Bytes/sec: {:.2}", bytes_per_sec);
println!(" Events/sec: {:.2}", events_per_sec);
println!(" Total stats: {}", current_stats);
```

```
self.last_stats = current_stats;
self.last_check = now;
}
}
```

실제 사용 사례

1. P2P 채팅 애플리케이션

```
use libp2p::{
gossipsub, mdns, noise, tcp, yamux,
swarm::{NetworkBehaviour, SwarmBuilder, SwarmEvent},
};
use tokio::{io::{self, AsyncBufReadExt}, select};

#[derive(NetworkBehaviour)]
struct ChatBehaviour {
gossipsub: gossipsub::Behaviour,
mdns: mdns::tokio::Behaviour,
}

struct ChatApp {
```

```
swarm: libp2p::Swarm<ChatBehaviour>,

topic: gossipsub::IdentTopic,

}



impl ChatApp {

async fn new() -> Result<Self, Box<dyn std::error::Error>> {

let swarm = SwarmBuilder::with_new_identity()

.with_tokio()

.with_tcp(

tcp::Config::default(),

noise::Config::new,

yamux::Config::default(),

)?

.with_behaviour(|key| {

let gossipsub_config = gossipsub::ConfigBuilder::default()

.heartbeat_interval(Duration::from_secs(10))

.validation_mode(gossipsub::ValidationMode::Strict)

.build()

.expect("Valid config");



let gossipsub = gossipsub::Behaviour::new(

gossipsub::MessageAuthenticity::Signed(key.clone()),

gossipsub_config,
```

```
)?;

let mdns = mdns::tokio::Behaviour::new(
mdns::Config::default(),
key.public().to_peer_id(),
)?;

Ok(ChatBehaviour { gossipsub, mdns })
})?
.build();

let topic = gossipsub::IdentTopic::new("chat");

Ok(ChatApp { swarm, topic })
}

async fn run(&mut self) -> Result<(), Box<dyn std::error::Error>> {
// 토픽 구독
self.swarm.behaviour_mut().gossipsub.subscribe(&self.topic)?;

// 리스닝 시작
self.swarm.listen_on("/ip4/0.0.0.0/tcp/0".parse()?)?;
```

```
let mut stdin = io::BufReader::new(io::stdin()).lines();


println!("Chat application started. Type messages to send:");


loop {
select! {
// 사용자 입력 처리
Ok(Some(line)) = stdin.next_line() => {
if let Err(e) = self.swarm
.behaviour_mut()
.gossipsub
.publish(self.topic.clone(), line.as_bytes()) {
println!("Failed to publish message: {e:?}");
}
}


// Swarm 이벤트 처리
event = self.swarm.select_next_some() => {
self.handle_swarm_event(event).await;
}
}
}
}
```

```
async fn handle_swarm_event(&mut self, event: SwarmEvent<ChatBehaviourEvent>) {
match event {
SwarmEvent::Behaviour(ChatBehaviourEvent::Mdns(mdns::Event::Discovered(list))) =
for (peer_id, _multiaddr) in list {
println!("Discovered peer: {peer_id}");
self.swarm.behaviour_mut().gossipsub.add_explicit_peer(&peer_id);
}
}
SwarmEvent::Behaviour(ChatBehaviourEvent::Mdns(mdns::Event::Expired(list))) => {
for (peer_id, _multiaddr) in list {
println!("Peer expired: {peer_id}");
self.swarm.behaviour_mut().gossipsub.remove_explicit_peer(&peer_id);
}
}
SwarmEvent::Behaviour(ChatBehaviourEvent::Gossipsub(gossipsub::Event::Message {
propagation_source: peer_id,
message,
..
})) => {
let msg = String::from_utf8_lossy(&message.data);
println!("[{peer_id}]: {msg}");
}
```

```
SwarmEvent::NewListenAddr { address, .. } => {
println!("Listening on {address}");
}
_ => {}
}
}
}
}

#[tokio::main]
async fn main() -> Result<(), Box<dyn std::error::Error>> {
let mut app = ChatApp::new().await?;
app.run().await
}
```

2. 분산 파일 공유 시스템

```
use libp2p::{
kad, request_response,
swarm::{NetworkBehaviour, SwarmBuilder},
StreamProtocol,
};
use serde::{Deserialize, Serialize};
use std::collections::HashMap;
```

```rust
#[derive(Debug, Clone, Serialize, Deserialize)]
enum FileRequest {
Get { hash: String },
Put { hash: String, data: Vec<u8> },
List,
}

#[derive(Debug, Clone, Serialize, Deserialize)]
enum FileResponse {
File { data: Vec<u8> },
Hash { hash: String },
FileList { hashes: Vec<String> },
Error { message: String },
}

#[derive(NetworkBehaviour)]
struct FileShareBehaviour {
kad: kad::Behaviour<kad::store::MemoryStore>,
request_response: request_response::Behaviour<request_response::cbor::Codec<File
}

struct FileShareNode {
```

```
swarm: libp2p::Swarm<FileShareBehaviour>,

local_files: HashMap<String, Vec<u8>>,

}



impl FileShareNode {

async fn new() -> Result<Self, Box<dyn std::error::Error>> {

let swarm = SwarmBuilder::with_new_identity()

.with_tokio()

.with_tcp(

tcp::Config::default(),

noise::Config::new,

yamux::Config::default(),

)?

.with_behaviour(|key| {

let peer_id = key.public().to_peer_id();

let store = kad::store::MemoryStore::new(peer_id);

let kad = kad::Behaviour::new(peer_id, store);



let protocols = std::iter::once((

StreamProtocol::new("/file-share/1.0.0"),

request_response::ProtocolSupport::Full,

));

let request_response = request_response::Behaviour::with_codec(
```

```
request_response::cbor::Codec::default(),
protocols,
request_response::Config::default(),
);


Ok(FileShareBehaviour { kad, request_response })
})?
.build();


Ok(FileShareNode {
swarm,
local_files: HashMap::new(),
})
}


async fn add_file(&mut self, data: Vec<u8>) -> String {
use std::collections::hash_map::DefaultHasher;
use std::hash::{Hash, Hasher};


let mut hasher = DefaultHasher::new();
data.hash(&mut hasher);
let hash = format!("{:x}", hasher.finish());
```

```
self.local_files.insert(hash.clone(), data);

// DHT에 파일 해시 저장

let key = kad::record::Key::new(&hash);

let record = kad::Record::new(key, hash.as_bytes().to_vec());

self.swarm.behaviour_mut().kad.put_record(record, kad::Quorum::One).ok();



hash

}



async fn request_file(&mut self, peer_id: PeerId, hash: String) {

let request = FileRequest::Get { hash };

self.swarm.behaviour_mut().request_response.send_request(&peer_id, request);

}



async fn run(&mut self) -> Result<(), Box<dyn std::error::Error>> {

self.swarm.listen_on("/ip4/0.0.0.0/tcp/0".parse()?)?;



loop {

match self.swarm.select_next_some().await {

SwarmEvent::Behaviour(FileShareBehaviourEvent::RequestResponse(

request_response::Event::Message { message, .. }

)) => {

match message {
```

```
request_response::Message::Request { request, channel, .. } => {
let response = self.handle_file_request(request).await;
self.swarm.behaviour_mut().request_response
.send_response(channel, response).ok();
}
request_response::Message::Response { response, .. } => {
self.handle_file_response(response).await;
}
}
}
SwarmEvent::Behaviour(FileShareBehaviourEvent::Kad(kad::Event::OutboundQueryProg
result: kad::QueryResult::GetRecord(Ok(kad::GetRecordOk::FoundRecord(record))),
..
})) => {
println!("Found file record: {:?}", record);
}
SwarmEvent::NewListenAddr { address, .. } => {
println!("File share node listening on {address}");
}
_ => {}
}
}
}
```

```
async fn handle_file_request(&self, request: FileRequest) -> FileResponse {
match request {
FileRequest::Get { hash } => {
if let Some(data) = self.local_files.get(&hash) {
FileResponse::File { data: data.clone() }
} else {
FileResponse::Error { message: "File not found".to_string() }
}
}
FileRequest::Put { hash, data } => {
// 실제 구현에서는 검증 로직 추가
FileResponse::Hash { hash }
}
FileRequest::List => {
let hashes = self.local_files.keys().cloned().collect();
FileResponse::FileList { hashes }
}
}
}

async fn handle_file_response(&self, response: FileResponse) {
match response {
```

```
FileResponse::File { data } => {
println!("Received file with {} bytes", data.len());
}
FileResponse::Error { message } => {
println!("File request error: {message}");
}
_ => {}
}
}
}
```

결론

이 문서는 Rust libp2p의 Swarm 컴포넌트를 다양한 시나리오에서 설정하고 사용하는 방법을 포괄적으로 다루었습니다. 기본적인 설정부터 고급 최적화, 오류 처리, 실제 애플리케이션 구현까지 실용적인 예제를 통해 Swarm의 강력한 기능을 활용하는 방법을 보여주었습니다.

주요 포인트:

- **모듈식 설계**: 다양한 전송 프로토콜과 네트워크 동작을 조합 가능
- **이벤트 기반 아키텍처**: 비동기 이벤트 처리를 통한 효율적인 네트워크 관리
- **확장성**: 연결 제한, 성능 최적화를 통한 대규모 네트워크 지원
- **복원력**: 자동 복구 메커니즘과 오류 처리를 통한 안정적인 운영

이러한 예제들을 참고하여 자신의 P2P 애플리케이션에 맞는 Swarm 설정을 구성할 수 있습니다.